Give me some industry-standard ways to separate bootstrap classes from your own custom classes.

A common challenge with Bootstrap is making it clear which classes come from Bootstrap and which are part of your application's design system. There isn't a single official standard, but several widely adopted approaches are used in professional projects.

1. Prefix all custom classes (very common)

Add a consistent prefix to every custom class.

HTML

```html
<div class="card app-user-card">
  ...
</div>
```

CSS

```css
.app-user-card {
  border-left: 4px solid #0d6efd;
}
```

Common prefixes:

```
.app-
site-
c-
u-
js-
```

Example:

```html
<button class="btn btn-primary app-save-button">
  Save
</button>
```

Advantages

- Immediately distinguishes Bootstrap from custom code.
- Reduces naming collisions.
- Scales well for large teams.

---

2. Use BEM naming for custom classes

Keep Bootstrap utility/layout classes, but structure your own components with BEM.

```html
<div class="card user-profile">
  <div class="user-profile__header">
    ...
  </div>
</div>
```

```css
.user-profile {}
.user-profile__header {}
.user-profile--compact {}
```

This makes it obvious that:

```
card
row
col-md-6
text-center
```

are Bootstrap, while:

```
user-profile
user-profile__header
```

are application-specific.

---

3. Separate layout and component classes

A popular convention is:

```html
<div class="row">
  <div class="col-md-6">
    <div class="customer-card">
      ...
    </div>
  </div>
</div>
```

Bootstrap handles:

```
row
col-md-6
```

Your code handles:

```
customer-card
```

This keeps responsibilities clear:

- Bootstrap → grid, spacing, utilities.
- Custom CSS → business-specific components.

---

4. Namespace custom components

Large projects often use component namespaces.

```html
<div class="c-user-card">
  ...
</div>
```

```css
.c-user-card {}
.c-user-card__title {}
```

Example convention:

```css
.c-  /* component */
.l-  /* layout */
.u-  /* utility */
.is- /* state */
.js- /* JavaScript hooks */
```

```html
<div class="c-user-card is-active">
  ...
</div>
```

This comes from methodologies like ITCSS and SUIT CSS.

---

5. Use Bootstrap utilities only, custom classes for components

A modern Bootstrap approach:

```html
<div class="user-card p-3 mb-4 d-flex align-items-center">
  ...
</div>
```

Bootstrap:

```
p-3
mb-4
d-flex
align-items-center
```

Custom:

```
user-card
```

This is arguably the most common pattern in current Bootstrap projects.

---

6. Keep JavaScript classes separate

Avoid styling classes being used as JS selectors.

```html
<button class="btn btn-primary js-save-user">
  Save
</button>
```

```js
document.querySelector('.js-save-user');
```

This prevents CSS refactors from breaking JavaScript.

Recommended approach

For most modern Bootstrap applications:

```html
<div class="card app-user-card p-3">
  <h3 class="app-user-card__title">
    John Smith
  </h3>
</div>
```

Where:

- Bootstrap classes handle layout and utilities:

  ```
  card p-3
  ```

- Custom classes are prefixed:

  ```
  app-user-card
  app-user-card__title
  ```

- Custom classes follow BEM when components become complex.

This combination is easy to read, scales well, and is commonly used in production applications.

"what are the best ways to retrieve data from HTML forms using JS?"

The "best" way depends on the type of form and what you're doing with the data, but these are the most common industry-standard approaches.

1. `FormData` (recommended for most forms)

For collecting all form fields at once, `FormData` is usually the cleanest solution.

```html
<form id="userForm">
  <input name="firstName" value="John">
  <input name="email" value="john@example.com">
  <button type="submit">Submit</button>
</form>
```

```js
const form = document.getElementById('userForm');

form.addEventListener('submit', (e) => {
  e.preventDefault();

  const formData = new FormData(form);

  console.log(formData.get('firstName'));
  console.log(formData.get('email'));
});
```

Convert directly to an object:

```js
const data = Object.fromEntries(new FormData(form));

console.log(data);
/*
{
  firstName: "John",
  email: "john@example.com"
}
*/
```

Advantages

- Reads the whole form automatically.
- Works with file uploads.
- Less repetitive code.
- Great for API requests.

---

2. Access fields through the form element

Useful when working with specific fields.

```js
const form = document.getElementById('userForm');

const firstName = form.elements.firstName.value;
const email = form.elements.email.value;
```

Or:

```js
const firstName = form.elements['firstName'].value;
```

Advantages

- Fast and readable.
- Doesn't require extra selectors.
- Good when only a few fields are needed.

---

3. Query selectors

Common when fields need custom handling.

```js
const firstName =
  document.querySelector('[name="firstName"]').value;

const email =
  document.querySelector('[name="email"]').value;
```

Or scoped to a form:

```js
const firstName =
  form.querySelector('[name="firstName"]').value;
```

Advantages

- Flexible.
- Works well with dynamic forms.

Disadvantages

- More verbose.
- Easy to accidentally select the wrong element.

---

4. Destructure from `FormData` into an object

A common modern pattern:

```js
const data = Object.fromEntries(
  new FormData(form)
);

const { firstName, email } = data;
```

This is very convenient when sending JSON:

```js
fetch('/api/users', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(data)
});
```

---

5. For checkboxes and radio buttons

Checkbox

```html
<input type="checkbox" name="newsletter">
```

```js
const checked =
  form.elements.newsletter.checked;
```

Radio group

```html
<input type="radio" name="role" value="admin">
<input type="radio" name="role" value="user">
```

```js
const role =
  new FormData(form).get('role');
```

`FormData` handles radio groups very nicely.

---

6. For multiple selections

```html
<select name="tags" multiple>
  <option value="js">JS</option>
  <option value="css">CSS</option>
</select>
```

```js
const tags = formData.getAll('tags');
```

Result:

```
["js", "css"]
```

Typical modern pattern

For most applications today:

```js
form.addEventListener('submit', (e) => {
  e.preventDefault();

  const data = Object.fromEntries(
    new FormData(e.target)
  );

  console.log(data);

  // send to API
  fetch('/api', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify(data)
  });
});
```

This approach is concise, works with most form controls, and is the pattern you'll frequently see in modern vanilla JavaScript applications.

Generate a regex for the JS .replace() method that removes all whitespace from a string

Use the regular expression:

```
/\s+/g
```

With `.replace()`:

```js
const result = str.replace(/\s+/g, '');
```

Example:

```js
const str = ' Hello \t world \n ';
const result = str.replace(/\s+/g, '');

console.log(result); // "Helloworld"
```

Explanation

- `\s` matches any whitespace character:
  - spaces
  - tabs (`\t`)
  - newlines (`\n`)
  - carriage returns (`\r`)
  - other Unicode whitespace
- `+` matches one or more consecutive whitespace characters.
- `g` (global flag) replaces all matches in the string.

If you only want to remove ordinary space characters and leave tabs/newlines intact, use:

```
/ /g
```

But for "remove all whitespace," `str.replace(/\s+/g, '')` is the standard solution.

I am using leaflet to create a map that marks the location of fruit trees on a map. My current solution changes the marker size based on the map zoom (larger markers when zoomed, smaller markers when zoomed out). The current issue is the all markers are destoryed when the size is changed. This is inefficent and makes the map crash. How can I render the markers once but dynamically change the size? here is my current solution for reference:

```
renderTreeMarker(tree, zoom) {
    const [LNG, LAT] = tree.geometry.coordinates
    const TREECOMMONNAME =
```

Show more

The main issue is that you're **creating entirely new marker instances on every zoom change**. Leaflet has to remove thousands of markers, create thousands of new ones, rebind popups, reattach event listeners, and redraw everything. That's why performance collapses.

A better approach is:

1. Create each marker **once**.
2. Store references to the markers.
3. Listen for `zoomend`.
4. Update the existing marker's icon or radius instead of recreating it.

Option 1: Use `setRadius()` for circle markers

If you're using `L.circleMarker`, Leaflet supports changing the radius directly:

```js
marker.setRadius(newRadius);
```

Example:

```js
this.map.on('zoomend', () => {
    const zoom = this.map.getZoom();

    this.treeMarkers.forEach(marker => {
        marker.setRadius(
            zoom <= 13 ? 1 :
            zoom === 14 ? 2 :
            4
        );
    });
});
```

No marker destruction occurs.

Option 2: Use `setIcon()` for icon markers

For `L.marker`, create a new icon and assign it:

```js
marker.setIcon(newIcon);
```

Example:

```js
this.map.on('zoomend', () => {
    const zoom = this.map.getZoom();

    this.treeMarkers.forEach(marker => {
        const size =
            zoom === 15 ? 24 :
            zoom === 16 ? 36 :
            zoom >= 17 ? 44 :
            16;

        marker.setIcon(
            L.icon({
                iconUrl: marker.options.treeIcon,
                iconSize: [size, size],
                iconAnchor: [size / 2, size / 2]
            })
        );
    });
});
```

This updates the DOM node rather than recreating the marker.

Option 3: Best Performance — Use CSS scaling

Since your icons are SVGs, you can often avoid calling `setIcon()` entirely.

Create all markers with a fixed icon size:

```js
const icon = L.icon({
    iconUrl: MARKER_ICON,
    iconSize: [44, 44],
    iconAnchor: [22, 22],
    className: 'tree-marker'
});
```

Then change their scale with CSS:

```css
.tree-marker {
    transition: transform 0.1s linear;
}
```

On zoom:

```js
this.map.on('zoomend', () => {
    const zoom = this.map.getZoom();

    let scale = 0.25;

    if (zoom === 15) scale = 0.55;
    if (zoom === 16) scale = 0.8;
    if (zoom >= 17) scale = 1;

    document.documentElement.style.setProperty(
        '--tree-scale',
        scale
    );
});
```

```css
.tree-marker {
    transform: scale(var(--tree-scale, 1));
}
```

This is usually much faster because:

- No marker recreation.
- No icon recreation.
- No popup rebinding.
- Browser handles scaling via GPU.

Option 4: Use a custom `DivIcon` and CSS classes

Another common Leaflet pattern:

```js
marker.setIcon(
    L.divIcon({
        className: `tree-marker zoom-${zoom}`
    })
);
```

and control appearance entirely via CSS.

Additional issue in your code

You're repeatedly duplicating:

```js
marker.bindPopup(...)
marker.on("click", ...)
```

for every zoom level.

Those should only be attached **once when the marker is created**.

For example:

```js
const marker = L.marker([LAT, LNG], {
    icon: initialIcon
});

marker.bindPopup(MARKER_POPUP_CONTENT);

marker.on("click", () => {
    this.map.panTo(marker.getLatLng());

    setTimeout(() => {
        marker.openPopup();
    }, 100);
});
```

Then on zoom:

```js
marker.setIcon(newIcon);
```

and nothing else.

For a fruit-tree map with potentially hundreds or thousands of markers, the most efficient architecture is:

- Create markers once.
- Store them in an array or `LayerGroup`.
- Update size via `setRadius()` or CSS transforms.
- Never recreate markers on every zoom event.
- If you have very large datasets (5k+ trees), consider using a canvas renderer (`L.canvas()`) or a clustering plugin such as `Leaflet.markercluster`.